# NanoBook: Viewer Report

**A live L1 watchlist that the engine never knows about** · 26 October 2026 · C++23, GCC 14 / Clang 19, FTXUI 6.1.9, Linux · Built on the `ui` branch after `v1.0.0`; the publisher's cost measured on bare metal

## 1. Summary

The roadmap ended at phase 7 (`v1.0.0`). Plan §1 allowed one extra after it: a **live view** in a **separate process**, with one hard rule: the engine thread must never know it exists. This is that view: `nanobook_view`, a terminal watchlist of every stock's best bid and ask (L1), fed by the pipeline's publisher through **shared memory**.

| | |
|---|---|
| **Status** | **Complete**: all 7 steps of viewer design §8. Step 4, the publisher's cost, ran in one bare-metal session (§7.3) |
| **The engine** | **Untouched.** No file under `include/` or `src/` changed since `v1.0.0`; the full day's log has the published checksum (`748abedf40533b82`) with the viewer off, on, and on one thread (§6.1) |
| **The table** | One 64-byte slot per stock (65,536, one per locate), a **seqlock** each: the publisher never waits, a reader never blocks it (§5) |
| **Watching** | 60× the ITCH clock held to **59.9–60.4×**; about **1.3–1.4 M updates/s** through the open (§7.1) |
| **The publisher's cost** | Bare metal, 1–4 M records/s: writing the table, **nothing measurable**; with the viewer attached, p99 **+6 to +45 ns** (1–7%), not clearly beyond the noise. No fix (§7.3) |
| **Latency on screen** | p99 **31–54 µs at 1×**, **0.4–2.4 ms at 60×**: the difference is queueing behind sped-up bursts, not a fault (§7.2) |
| **Tests** | **819** in Release builds (800 with sanitizers): **29 new**, plus 4 layout tests in the viewer's own program (`nanobook_view_tests`) |
| **Planted bugs** | **12** deliberate bugs: **11 caught**; one weak test found and fixed on the way; the one miss (a memory-order bug) explained in §6.3 |

How to read this report:

- §2 explains the viewer in plain words, and §3 the plan.
- §4 covers what was built, and §5 the seqlock line by line.
- §6 shows how correctness was proven, §7 what watching the day showed and what the viewer costs.
- §8 lists the decisions and what changed along the way, and §9 what's next.

All numbers are from WSL2 on an Intel Core i7-14650HX (24 threads), kernel 6.18.54.1-microsoft-standard-WSL2, GCC 14.2: **not for publishing** (rule 20). None of them is a published latency claim; they say what the viewer shows and why. **Except §7.3**, the publisher's cost: measured on the same laptop on bare metal, with isolated cores, so it can be published.

## 2. What the viewer is, in plain words

The pipeline already had three workers on an assembly line (phase 5): the **feed** reads the Nasdaq file, the **engine** keeps the books, the **publisher** logs every change to a stock's best bid and ask. The viewer adds a **window onto the publisher's desk**, in another room.

Three ways to look at an order book:

| Level | What it is | In NanoBook |
|---|---|---|
| **L3** | Every resting order, one by one | The books themselves (ITCH is an order-by-order feed) |
| **L2** | Total shares at each price | Computed by `depth()`, but it allocates, so only tests and plots use it |

| Level | What it is | In NanoBook |
|---|---|---|
| **L1** | Best bid and best ask, with their sizes | What the engine publishes: one `TopOfBook` after every order message |

The viewer shows **L1**, because L1 is what already reaches the publisher. Anything deeper would mean more work on the engine thread for every update.

The problem a viewer has to solve: the pipeline publishes **over a million updates a second**, and a screen redraws ten times a second. The viewer can't see every update, and must not try: if the publisher waited for it, the engine would stall behind the publisher. So it reads **the latest value of each stock**, whenever it likes. Updates between two redraws are simply skipped.

Picture a **departures board**. Each flight has one line that the airline overwrites when the gate changes. Nobody queues to read it, and the airline never waits for readers. If you glance up while a line is being rewritten, you look again a moment later.

---

## 3. The plan and where it stands

| Step | Deliverable | State |
|---|---|---|
| 1 | Design note (`docs/viewer-design.md`) | Approved 2026-10-26, every decision as proposed |
| 2 | The L1 table in shared memory, with tests | Done (§4.1) |
| 3 | `nanobook_pipeline --view NAME`: the publisher writes the table | Done (§4.2); checksum unchanged (§6.1) |
| 4 | The publisher's cost, before and after (rule 22) | Done on bare metal (§7.3), after step 6 (the user's choice) |
| 5 | `--speed N`: pacing by the ITCH clock | Done (§4.3) |
| 6 | `nanobook_view` on FTXUI, CI job | Done (§4.4) |
| 7 | README "Watch it run", this report | Done |

---

## 4. What was built

### 4.1 The table (`support/l1_table.hpp`, `support/l1_mapping.hpp`)

| Part | Size | Written |
|---|---|---|
| Header: magic, layout version, run id, state, the counter's ticks per second | 1 cache line | When the table is made, and at the run's start and end |
| Header: updates so far, last ITCH time | 1 cache line | Every update |
| Names, one 8-character symbol per locate | 512 KiB | Once, before the run starts |
| Slots, one per locate | 64 bytes × 65,536 = 4 MiB | Every update of that stock |
| Latency histogram, the recorder's buckets | 2,432 × 8 bytes = 19 KiB | Every paced update |

About 4.6 MiB in all. A locate is a `uint16_t`, so every locate has a slot and finding one is just an index, never a lookup.

`L1Mapping::create(name)` makes it under `/dev/shm/<name>` (POSIX shared memory: a file in RAM, never written to disk) and builds the table in it with a placement `new`. That writes every byte, so every page is touched before the run and the publisher never takes a page fault on it. `open(name)` maps it **read-only**: a viewer can't write it, even by mistake (a store would crash the viewer, not corrupt the table). The magic number is stored last, with release, so a reader who sees it sees a finished table.

The table lives in `support/`, next to the publisher's other sink (`binary_log.hpp`), not in `include/`: the design note said `include/nanobook/view/`, but the table reuses the latency recorder's buckets from `support/`, and the engine's headers must not depend on test support.

### 4.2 The publisher's new sink (`nanobook_pipeline --view NAME`)

1. Before the run: make the table and copy every stock's name into it, from the directory the tool already reads to size the books. Nothing new crosses ring 2.
2. Per update, after the log: write the stock's slot, bump the header's count and ITCH time, and in a paced run add the update's latency to the histogram.

Without `--view`, the sink is **the code it was before**, not a branch that skips the view: the latency run's sink is one template compiled twice (`std::bool_constant<kView>` and `if constexpr`). Step 4's "before" is then the real before.

### 4.3 Pacing by the ITCH clock (`ClockPacer`, `--speed N`)

Unpaced, the whole day takes about 100 s: too fast to watch. The phase 5 `Pacer` sends a fixed number of records per second, which flattens the day: the open's burst looks like lunch. `ClockPacer` keeps the day's shape. A record stamped t ns after `--from` is due t / speed after the start:

```
ticks = (t / 10^9) x ticks_per_second  +  (t mod 10^9) x ticks_per_second / 10^9
due   = max(last due, start + ticks / speed)
```

- **From the start every time**, never by adding gaps, so no rounding error builds up.
- **Split into whole seconds and the rest:** a whole day is 5.76 × 10^13 ns, times 5 × 10^9 ticks a second is 2.9 × 10^23, far past 2^64 (1.8 × 10^19). Split, the larger product stays below 2^64 up to an 18 GHz counter.
- **Never back:** a record whose ITCH time is earlier than the last one's is due at once.
- **Stamped with the time it was due** (rule 18, coordinated omission), exactly like `Pacer`: if the books fall behind, the wait shows up as latency.

### 4.4 The viewer (`nanobook_view`)

Split in three, so that most of it can be tested without a terminal:

| File | Job | Tested by |
|---|---|---|
| `support/watchlist.hpp` | The numbers: reads ~9,000 named slots, rates over 1 s windows, latency percentiles, sort, filter, pins | 10 tests in the main suite |
| `tools/view_render.hpp` | The layout: the health lines, one fixed-width line per stock, the keys | 4 tests that draw frames off-screen and read them back as text |
| `tools/nanobook_view.cpp` | The loop: a 100 ms timer, the keys, re-attaching to a newer run | By hand, on the real day (§7.1) |

- **Rates change once a second**, from 1 s windows: ten noisy values a second are harder to read.
- **Latency is the last second's**, from the difference between the histogram's bucket counts then and now; the p-th percentile is the bucket holding the sample at rank ⌈p × N⌉, reported as that bucket's lower edge (at most 1.6% low).
- **Every table read happens on the UI thread.** The timer thread only posts an event.
- **Start it before or after the pipeline**, and leave it open: once a second it looks for the name again and switches to a newer run's table.

FTXUI is pinned at **v6.1.9** and checked by SHA256, like GoogleTest. A v7 exists (June 2026); v6's API is the one explained here. The viewer builds only with `-DNANOBOOK_BUILD_VIEWER=ON`, so the default build and `reproduce.sh` download nothing more. CI builds it in its own job, and the clang-tidy job now covers it.

---

## 5. The seqlock, line by line

Each slot has a counter, `seq`: **even** while the slot is stable, **odd** while it is being written.

```
write (publisher only)                    read (any reader, any process)
  s = seq                (relaxed)          s1 = seq               (acquire)
  seq = s + 1            (relaxed)          if s1 is odd: busy, try again later
  7 fields               (release)          7 fields               (acquire)
  seq = s + 2            (release)          s2 = seq               (relaxed)
                                            if s2 != s1: torn, throw the copy away
```

Why each order:

- **The field stores are release:** nothing before them (the odd `seq`) can be seen after them. A reader that saw even one new field must then also see the odd `seq`, so its second `seq` differs from its first, and it throws the copy away.
- **The last `seq` store is release**, and the reader's first `seq` load is acquire: a reader that sees the even value sees every field written before it.
- **The reader's field loads are acquire,** so the second `seq` load can't be done before them.
- **Only the publisher writes,** so it bumps counters with a plain load and store. A `fetch_add` would be a locked instruction for no reason.

Two choices worth defending in an interview:

- **Every field is a `std::atomic<std::uint64_t>`.** A seqlock over plain fields is a data race: undefined behaviour in C++, even though the torn copy is thrown away. On x86-64 these release stores and acquire loads are still plain `mov`s; they only stop the compiler reordering.
- **No standalone fences.** The textbook seqlock uses `atomic_thread_fence`. ThreadSanitizer can't follow fences, and GCC warns about them under `-fsanitize=thread`, which `-Werror` turns into a failed build. Release stores and acquire loads say the same thing in a form TSan understands.

Atomics shared between processes must be **lock-free** (a lock hidden inside one process's atomic means nothing to another process): `static_assert(std::atomic<std::uint64_t>::is_always_lock_free)`.

---

## 6. How correctness was proven

### 6.1 The engine is untouched

The full day through `nanobook_pipeline --checksum`, three ways:

| Run | Updates | Log checksum | Wall time (one run each, not a benchmark) |
|---|---|---|---|
| Three threads, no view | 363,118,215 | `748abedf40533b82` | 101.6 s |
| Three threads, `--view` | 363,118,215, all written to the table | `748abedf40533b82` | 107.7 s |
| One thread, `--view` | 363,118,215, all written to the table | `748abedf40533b82` | 125.8 s |

The checksum is phase 5's published one. The table left behind was then read by an independent **Python decoder**, sharing no code with the C++: magic `NBVIEW01`, state Done, 363,118,215 updates; AAPL, SPY and MSFT empty at 20:00 (right: every order is gone after the close); and each slot's `seq` exactly twice its update count (two `seq` stores per write).

## 6.2 Tests

| File | Tests | What they check |
|---|---|---|
| `test_l1_table.cpp` | 11 | The layout; a write read back exactly; `publish()`; a busy slot refused; latency buckets and the clamp; the run's state; **torn reads under fire**; shared memory between two processes (`fork`) |
| `test_pipeline.cpp` | 1 | The real three-thread pipeline writing the table while another thread reads it: every stock's slot ends with its last update and count |
| `test_pacing.cpp` | 7 | `ClockPacer` with the fake clock: the warm-up, 1× and 10×, never back, the settle time, a whole day at 5 GHz exact, stamped on schedule when late |
| `test_watchlist.cpp` | 10 | Formatting, spreads, percentiles from buckets, rates over a window, sorting, filtering, pins, a busy slot keeping its last value, a new run starting over |
| `test_view_render.cpp` | 4 | Column names line up with a stock's line; a frame shows the run and the stocks, nothing cut off at 108 columns; only what fits is drawn and the cursor stays visible; a waiting viewer says why |

The torn-read test: one thread rewrites a slot 5 million times with quotes whose seven fields are all the same number, while the test reads it as fast as it can. Every read accepted must have seven equal fields, and must never go back in time. In the Release build it accepted 917,950 reads and refused 1,094,141 (the writer was in the middle), with no torn read. It runs under ASan, UBSan and TSan in CI.

## 6.3 Planted bugs: do the tests have teeth?

Each bug was planted alone, the tests run, the file restored.

| # | Planted bug | Caught by |
|---|---|---|
| 1 | The reader skips its second `seq` check | The torn-read test |
| 2 | The writer never marks the slot busy (no odd `seq`) | The torn-read test |
| 3 | The reader accepts an odd `seq` | The busy-slot test and the torn-read test |
| 4 | `publish()` doesn't count the stock's updates | The publish test and the pipeline test |
| 5 | A latency lands one bucket low | The latency bucket test |
| 6 | `ClockPacer`'s schedule can go back | The never-back test and the whole-day test |
| 7 | `ClockPacer` multiplies naively (`t × ticks / 10^9`) | The whole-day test (it overflows) |
| 8 | Percentile ranks round down, not up | The percentile test (p99.9 of 100 samples) |
| 9 | Rates never open a new window | The rate-window test |
| 10 | The filter is case-sensitive | The filter test |
| 11 | A new run keeps the old run's pins | The new-run test, **after it was fixed** |
| 12 | The field stores are relaxed, not release | **No test** |

Bug 11 first got through. The test pinned locate 2 in the old run, but the new run's only stock was at locate 1, so a leftover pin could never show. It now pins the locate the new run reuses.

Bug 12 got through, and no test on this machine can catch it:

- **On x86 the CPU never reorders a store with another store** (total store order), so relaxed and release stores give the same instructions in the same order. Only the compiler may reorder relaxed stores, and GCC didn't here.
- **ThreadSanitizer reports data races, and there are none:** every field is atomic, so a wrong order between atomics is no race. (Phase 5's ring was different: its slots are plain memory, so a wrong order there was a race, and TSan caught it.)

So the memory orders rest on the argument in §5, not on a test. On a CPU that reorders stores (ARM), bug 12 would let a reader accept a torn quote.

---

# 7. What watching it showed

## 7.1 The real day at 60×

`nanobook_pipeline FILE --view nanobook --speed 60 --from 09:30:00`, with the viewer open:

| | |
|---|---|
| **Start-up** | About 35 s: the counting pass over the 11 GB file, then 04:00–09:30 unpaced |
| **Speed held** | 59.9–60.4 ITCH seconds per real second, sampled every 5 s |
| **Updates, 09:30–09:50** | 1.16–1.43 M a second |
| **Busiest stocks at 09:37** | QQQ ~17,700 updates/s, SPY ~15,600, AMD ~14,800, AAPL ~11,800 |
| **Keys** | Pin, filter (any case), sort and quit all worked; a pinned stock stayed on top under every sort |

## 7.2 Why the latency tail is so long at 60×

At 60× the screen showed p50 ~0.7 µs but p99 0.4–2.4 ms, about 30 times phase 5's WSL2 p99. The same command at 1×, sampled every 5 s from 09:30:

| Speed | p50 | p99 | p99.9 |
|---|---|---|---|
| 1× | 0.6–3.4 µs | **31–54 µs** | 166 µs – 11.5 ms |
| 60× | ~0.7 µs | **0.4–2.4 ms** | 0.8–2.5 ms |

The cause is the **bursts**:

- Nasdaq's busiest moments pack thousands of messages into a millisecond.
- At 60× that millisecond arrives in 17 µs: more than the pipeline can take, so updates queue.
- The feed stamps each record with the time it was **due** (rule 18), so the queueing counts.

That is the measurement working, not a fault. At 1× the p99 matches phase 5's WSL2 figure (65–72 µs at a fixed rate). The worst p99.9 values (8–11 ms) are the virtual machine: watch runs are not pinned, and threads yield after a short spin. Above 1× the screen now says "(sped-up bursts queue)", so nobody reads a 60× tail as the engine's.

So **the publisher's cost must be measured with `--latency --rate`, not `--speed`** (§7.3): a fixed rate below saturation, as in phase 5, measures the pipeline; a sped-up day measures the bursts.

## 7.3 What the viewer costs the pipeline (bare metal)

The question of step 4: does the viewer slow the engine's output down? Three ways, each at the phase 5 rates of 1, 2 and 4 M records/s:

- **off:** no `--view`, the code of `v1.0.0`;
- **table:** `--view` with nobody reading, the cost of writing the table;
- **viewer:** `--view` with `nanobook_view` attached on its own core, the cost of a reader too.

`scripts/run_viewer_session.sh` on bare metal (cores 8–13 isolated; feed, engine and publisher on 8, 10, 12; the viewer on 2), 10:00–10:05 of the day (5.6 M records), two rounds. 18 runs, all clean, ring 2 never full. Every viewer screen ended on "done": it was reading throughout.

p99 in ns, rounds 1 / 2 (in brackets the change from `off` in the same round):

| Rate | off | table | viewer |
|---|---|---|---|
| 1 M/s | 605 / 628 | 618 / 632 (+12 / +4) | 650 / 636 (+45 / +8) |
| 2 M/s | 632 / 626 | 625 / 639 (-7 / +13) | 642 / 637 (+9 / +11) |
| 4 M/s | 687 / 681 | 688 / 684 (+1 / +3) | 693 / 711 (+6 / +30) |

p50 and p99.9 tell the same story (all of them: `docs/perf-log.md`, 2026-10-26). `off` matches the release session's phase 5 numbers (630–686 ns), so the machine behaved as it did then.

How to read it. The same run twice differs by 5–20 ns: that's the **noise**.

- **Writing the table costs nothing measurable.** `table` lands on both sides of `off`, inside the noise. The publisher writes a few stores into a cache line that only it touches.
- **A reader may cost a few ns.** `viewer` is slower than `off` in all 6 pairs, at p50, p99 and p99.9 alike: +6 to +11 ns at p99 in four (1–2%), +30 and +45 in two (5–7%).
- **But it isn't proven.** The order was the same every time (`off`, `table`, then `viewer`), so the viewer always ran last of its three; anything that drifts inside a group (heat, say) would look the same. And the +45 sits next to the session's lowest `off`.

A likely mechanism, not tested: think of each slot as a **whiteboard** that the publisher keeps on its own desk. When the viewer reads a slot, the board is copied to the viewer's desk too, and the publisher has to take the copy back before its next write (a "read for ownership", tens of ns between cores). That happens at most once per slot per refresh, so only to a few updates: the right shape for a few ns at p99.

**Decision: no fix.** The rule, fixed before the session, called for a fix on the publisher's side only if p99 moved by more than the noise, and it hasn't clearly done so. The engine is untouched either way.

---

## 8. Decisions and what changed along the way

| Decision | Chose | Instead of | Because |
|---|---|---|---|
| V1 | A terminal UI (FTXUI) | A web page | Same language, toolchain and CI; no server; works over SSH. A web page can be one more reader of the same table |
| V2 | A latest-value table with a seqlock | A shared-memory ring, or UDP | A ring keeps every update, so a slow reader blocks the writer or loses records. A viewer wants only the latest |
| V3 | Names written at start-up | A new record type on ring 2 | The tool already reads the directory; the engine stays unchanged |
| V4 | 10 redraws a second | 30 or 60 | Looks live, costs little; one constant |
| V5 | ITCH-clock pacing (`--speed`) | Fixed rate only | A fixed rate flattens the day |
| V6 | This repository, viewer off by default | A separate repository | The viewer depends on the table's layout; one commit changes both |

What changed along the way, each recorded in the design note:

- **The table moved to** `support/` (§4.1), to reuse the recorder's buckets.
- **Fences became release stores and acquire loads** (§5): GCC's TSan warning would have failed the build.
- **Step 4 moved** to one bare-metal session after step 6, the user's choice.
- **Found on the stick:** `scripts/run_viewer_session.sh` was committed without its executable bit (exit 126), so it ran as `bash scripts/run_viewer_session.sh`. Fixed, and CI now checks every `scripts/*.sh`.
- **Found while watching:** the 60× latency tail (§7.2), now labelled on screen.
- **Found by the layout tests:** the latency line was cut off at 120 columns; it was shortened until a 108-column frame shows all of it.
- **Found by planting bugs:** the weak new-run test (§6.3).

---

## 9. What's next

1. **If the reader's cost is ever worth settling** (§7.3): a bare-metal session with the order rotated (each way first, second and third once), 4 rounds, and `perf c2c` during a viewer run to see whether the slots' cache lines bounce between cores. If they do and it matters, the fix is on the publisher's side (for example, write each stock's slot every N updates), never on the engine.

2. **Then maybe:** an L2 ladder for one chosen stock (engine-side work, so a measured experiment); a trade tape; a small web page reading the same table.